

For each of the sub-tasks, add a comment statement, at the beginning of the code using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

```
ln [1] : # Task 1.1
         Program code
```

Output:

Task 1.1

Write a function `task1_1(string)` that takes in the first row of MRZ as a string, extract and returns the full name of the passport holder in surname + given name format. [5]

Task 1.2

Use the two input strings below to test your function.

```
aline1 = "PASGPWONG<<KARA<YUN<EN<<<<<<<<<<<<<<<<<"
blinel1 = "PASGPNQ<<LINDA<ZEE<ZEE<<<<<<<<<<<<<<<<<"
```

Test your function with the two input strings above by calling it using the following statement:

```
print(task1_1(string) == expected)
```

The two statements should all print `True`.

For example:

```
print(task1_1(aline1) == "WONG KARA YUN EN")
```

Task 1.3

Write a function `task1_3(string)` to verify if a passport substring check digit is valid.

The function should:

- remove the check digit on the right-hand side of string
- calculate the check digit from the remaining string
- return a Boolean representing the comparison between the calculated check digit and the original removed digit. [8]

Task 1.4

Write a function `task1_4(fullstring)` that returns a Boolean.

The function should:

- takes the second row of the MRZ as a string, `fullstring`
- use your function from Task 1.3 to calculate all the check digits of `fullstring`
- return a Boolean representing the validity of `fullstring`

[3]

Use the two input strings below to test your function.

```
aline2 = "K0000000E4SGP7705038F2210300S7788888H<<<<98"  
bline2 = "X2000434N4SGP7706275F1008232S7788888H<<<<96"
```

Test your function with the following **two** calls:

- `print(task1_4(aline2))`
- `print(task1_4(bline2))`

[2]

Save your Jupyter notebook for Task 1.

- 2 A programmer is writing a program to implement a simple queue data structure using Object-Orientated Programming (OOP).

The superclass, `Queue`, is implemented using a fixed-size one-dimensional array. When five items have been added, this simple queue cannot accept any further items.

It will store the following properties and methods:

- `data`, a one-dimensional array that can contain five data items
- `head`, pointer pointing to the first element in the queue
- `tail`, pointer pointing to the next available location
- `is_empty()` returns a Boolean to represent if the queue is empty
- `is_full()` returns a Boolean to represent if the queue is full
- `peek()` returns an item from the front of the queue
- `enqueue(item)` add the parameter item to the rear of the queue
- `dequeue()` removes an item from the front of the queue
- `display()` shows clearly all items in the queue from front to rear.
 - For example:
 - Contents of queue:
 - Front 1 2 3 Rear

For each of the sub-tasks, add a comment statement, at the beginning of the code using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

```
In [1] : # Task 2.1
        Program code
```

Output:

Task 2.1

Write program code for the superclass `Queue`.

[8]

The subclass, `CircularQueue`, inherits from the `Queue` class. The space released after removing an item can be reused.

It has an addition property, `number_in_q`, to hold the number of items already in the queue

Task 2.2

Write program code for the subclass `CircularQueue`. Your code should demonstrate the concepts of polymorphism and inheritance.

[5]

Task 2.3

A print server is a shared network resource and processes print requests from different clients in a FIFO manner. The programmer would like to implement circular queue to manage scheduling of print requests to the print server.

Create an instance of your `CircularQueue` class and display the final contents of your queue after the following calls:

- `enqueue("Job21")`
- `dequeue()`
- `dequeue()`
- `enqueue("Job32")`
- `enqueue("Job33")`
- `enqueue("Job42")`
- `enqueue("Job12")`
- `enqueue("Job16")`
- `dequeue()`
- `dequeue()`
- `dequeue()`
- `enqueue("Job17")`

[2]

Save your Jupyter notebook for Task 2.

- 3 An event management company is organising an IT-immersion event where participants can sign up for their three game events.

The company wants to create a web app that will allow the participants to register for these events: Cybersecurity Escape Room, AI Laser Tag and Digital Forensics Capture the Flag. The company also requires a database to be designed for ease of data management. This database should be normalised.

At each sign up on the web app, the participant can only register for only one game event. Upon each sign up, the following information is recorded in the database table, `ParticipantsTable`:

- `ParticipantID`: a system-generated unique 6-digit ID number of the participant.
- `GameEvent`: the event the participant signs up: Cybersecurity Escape Room (CER), AI Laser Tag (ALT) or Digital Forensics Capture the Flag (DFC).
- `ParticipantName`: the participant's name.
- `ParticipantNickname`: the participant's nickname for the game event.

For the Cybersecurity Escape Room, the following extra information is recorded in the database table `CERTable`:

- `ParticipantSensitivity`: True if the participant has sensitivity to flashing lights, False if otherwise.

For the AI Laser Tag, the following extra information is recorded in the database table `ALTTable`:

- `ParticipantHeight`: the height of the participant in centimetres.
- `ParticipantWeight`: the weight of the participant in kilograms.

For the Digital Forensics Capture the Flag, the following extra information is recorded in the database table `DFCTable`:

- `ParticipantAge`: the age of the participant.

Task 3.1

Create an SQL file called `Task3_1_<your_name><CG><index_number>.sql` to show the SQL code to create the database `registration.db` with the above four tables.

`ParticipantsTable` must use the `ParticipantID` as its primary key. The other tables must refer to this `ParticipantID` as a foreign key.

Save your SQL code as `Task3_1_<your_name><CG><index_number>.sql` [5]

Task 3.2

Write a Python program and the necessary files to create a web application to allow the participants to register for the game events. You may use your own form design as you deem fit for the purpose.

The web application must:

- generate the `ParticipantID` for the user
- accept the user input from the web form for the rest of the input fields. You are not required to validate or verify the user input.
- update the necessary database tables in the `registration.db`.
- return a HTML document titled “Confirmation of Registration” to display a table containing the user’s details for the registered event. This document is generated upon the user clicking the Submit button.

Save your Python program as `Task3_2_<your_name><CG><index_number>.py`.

Save any additional files and subfolders in a parent folder named `Task3_2_<your_name><CG><index_number>`. [11]

Task 3.3

Test your web application by creating another submit button called Test. Upon clicking this Test button, the web application should return a HTML document that displays three tables, one for each game event, containing the necessary participants details.

Submit a registration form for each of the three game events to test your web application. Save the output of this test case as `Task3_3_<your_name><CG><index_number>.html`. [5]

Task 3.4

Write a SQL code to count the number of participants for the AI Laser Tag game event. Run this query.

Save this code as `Task3_4_<your_name><CG><index_number>.sql`. [3]